BDA400 / Data Science Tools and Techniques

Assignment 5

Technical Analysis using R, Development Phase

Repository link: https://github.com/donaldnyingifa/Assignment5-TechnicalAnalysis

Overview

This document explains the R implementation of nine technical analysis functions built for this assignment: SMA, EMA, MACD, standard deviation, linear regression, RSI, Stochastic RSI, crossover, and crossunder. Every function was written from scratch using only R's core language features, with no outside libraries, exactly as the assignment asked. Each one follows the template and pseudocode from the assignment brief, so the names, arguments, and return values line up with what the test file expects.

Each script lives on its own in the R_scripts folder, matching the naming convention from the templates (sma.R, ema.R, macd.R, and so on). A run_all_tests.R file is also included that sources every function and runs the sample data from the assignment so you can see all nine working in one go.

How to Run the Code

- Open R or RStudio and set your working directory to the R_scripts folder.
- Source the individual file for the function you want to test, e.g. source("sma.R").
- Some functions depend on others: macd.R sources ema.R, and stoch_rsi.R sources rsi.R and sma.R. Those source() calls are already included at the top of the relevant files, so you don't need to load dependencies manually.
- To see every function run at once with the assignment's sample data, source run_all_tests.R. It prints the result of each function to the console with a clear header.

Example:

```
setwd("path/to/R_scripts") source("run_all_tests.R")
```

Function Explanations

1. Simple Moving Average — sma(data, period)

The SMA smooths out a series of numbers by averaging every window of 'period' consecutive values, giving each point in that window the same weight. The function first checks that there's enough data to even form one window, then slides across the data one step at a time, summing each window and dividing by the period. The result is a vector that's shorter than the original data by period minus one, since the first full window only becomes available once you have that many points.

2. Exponential Moving Average — ema(data, period)

The EMA leans more heavily on recent values than the SMA does. It starts by working out a multiplier from the period (2 divided by period plus 1), which controls how much weight the newest point gets. The first EMA value is simply set equal to the first data point, since there's no earlier average to build from. From there, each new EMA value blends the current price with the previous EMA, weighted by that multiplier, so the average reacts quickly to price changes while still remembering the past.

3. Moving Average Convergence Divergence — macd(data, short_period, long_period, signal_period)

MACD is built directly on top of the EMA function. A short-period EMA and a long-period EMA are calculated from the same data, and subtracting the long EMA from the short EMA gives the MACD line, which shows momentum. That MACD line is then run through the EMA function again using the signal_period to produce the signal line, a smoothed version of the MACD line itself. The histogram is just the gap between the two, and it's often what traders watch most closely since it shrinks and grows as momentum builds or fades. The function returns all three as a named list.

4. Standard Deviation — stdev(data)

Standard deviation captures how spread out a set of numbers is. The function first works out the mean of the data, then finds how far each point sits from that mean. Those differences are squared, which stops positive and negative gaps from cancelling each other out, and the squared values are averaged to get the variance. Taking the square root of the variance brings everything back to the original units, giving the standard deviation. A larger result means the data swings more widely around its average.

5. Linear Regression — linreg(regressionSource, regressionLength, regressionOffset)

This function fits a straight line through a chosen slice of the data rather than always working across the entire series. regressionLength decides how many points are included in that slice, and regressionOffset lets you shift the slice away from the very end of the data if needed. Once the relevant subset is pulled out, the function assigns index values 1, 2, 3 and so on to represent time, works out the mean of the indices and the mean of the values, and uses the standard least squares formulas to compute the slope and intercept that minimize the distance between the line and the actual points. Predicted values are then generated for that same slice using the fitted line, and the slope, intercept, and predictions are returned together as a list.

6. Relative Strength Index — rsi(data, period)

RSI measures momentum by comparing the size of recent gains to the size of recent losses. The function starts by finding the difference between each consecutive pair of data points, then separates those differences into a gains vector and a losses vector. The average gain and average loss over the first period are calculated as a starting point, and every value after that rolls forward using Wilder's smoothing method, where the new average blends the previous average with the latest gain or loss. Dividing the average gain by the average loss gives the relative strength, which then feeds into the RSI formula. Because the first period values have nothing to compare against yet, they're left as NA.

7. Stochastic RSI — stoch_rsi(data, period, k_period, d_period)

Stochastic RSI applies the stochastic oscillator idea on top of RSI instead of on top of raw price. It first calculates RSI over the given period, then works out the lowest and highest RSI values seen so far and uses them to rescale every RSI value into a 0 to 1 range. That rescaled series is smoothed with the SMA function using k_period to produce the %K line, and %K is smoothed again with SMA using d_period to produce %D. The leading NA values that RSI naturally produces are dropped before this smoothing so the calculations stay clean.

8. Crossover — crossover(arr1, arr2)

This function flags the exact moment one line moves from being at or below a second line to strictly above it, which traders read as a possible uptrend signal. After confirming both arrays are the same length, it walks through the data starting at the second point and checks two things at once: is arr1 now greater than arr2, and was arr1 previously less than or equal to arr2. Only when both conditions are true does that point get marked "Up"; everything else is marked "None".

9. Crossunder — crossunder(arr1, arr2)

Crossunder is the mirror image of crossover. It looks for the point where arr1 drops from being at or above arr2 to strictly below it, which can hint at a possible downtrend. The logic is the same as crossover but flipped: it checks whether arr1 is now less than arr2 while it was greater than or equal to arr2 on the previous step, marking those points "True" and everything else "False".

Testing and Verification

Every function was tested using the sample data provided in the assignment brief and produced results consistent with the described formulas and pseudocode. For example, sma() on c(10, 12, 15, 20, 18, 22, 25, 24, 21) with period 3 correctly returns a smoothed series starting at 12.33, and crossover() applied to the two sample arrays correctly

flags a single "Up" event at the point where arr1 overtakes arr2. The run_all_tests.R script reproduces all of these checks in one place for easy review.